

Fully Online Retrieval Adaptation

Experiment Guideline

27 August 2026

1 Problem

Let $Z = (z_{t,u}) \in \mathbb{R}^{T \times U}$ be one time-series panel, where $t \in \{0, \dots, T-1\}$ is a date index and $u \in \{1, \dots, U\}$ is a series (called a user in the code). For a query date q , a lookback length L , and a forecast horizon H , define

$$x_{q,u} = (z_{q-L+1,u}, \dots, z_{q,u}) \in \mathbb{R}^L, \quad y_{q,u} = (z_{q+1,u}, \dots, z_{q+H,u}) \in \mathbb{R}^H.$$

The task is to predict every $y_{q,u}$. There is no train/validation/test split. At date q , a row dated r may be retrieved or fitted only if

$$r + H \leq q, \tag{1}$$

so its complete target $y_{r,u}$ is already observed. The experiment asks whether these causal rows improve a frozen forecasting model.

The CSV loader discovers `config.json` beside the selected dataset. Portable top-level values are overridden by the `online_adaptation` object and then by explicit run settings. For `drop_users`, null or omission inherits the preceding level, an empty list retains every CSV user, and a nonempty value replaces the preceding list. Exclusions are applied once while reading the CSV and the effective list is recorded in the run manifest.

2 Mathematical definition

2.1 Datastore, fitting dates, and queries

Let P be the dataset period: $P = 24$ for hourly data, $P = 7$ for daily data, and $P = 96$ for 15-minute data. Let S_{store} , S_{fit} , and S_q be the datastore, fitting, and evaluation-query strides. With periodic alignment, the latest causal datastore date for query q is

$$b_q = q - P \left\lceil \frac{H}{P} \right\rceil. \tag{2}$$

Without alignment, $b_q = q - H$. The available datastore-date grid is

$$\mathcal{D}_q^\infty = \{b_q - jS_{\text{store}} : j \in \mathbb{N}_0, b_q - jS_{\text{store}} \geq L - 1\}, \tag{3}$$

where $\mathbb{N}_0 = \{0, 1, 2, \dots\}$. Aligned runs require S_{store} to be a multiple of P .

Let N_{store} be the datastore-size setting and let M be its maximum number of dates:

$$M = \begin{cases} N_{\text{store}}, & \text{same-user retrieval,} \\ \lfloor N_{\text{store}}/U \rfloor, & \text{all-user or other-user retrieval.} \end{cases} \tag{4}$$

Thus cross-user retrieval requires $N_{\text{store}} \geq U$ and never keeps a partial oldest date. A rolling store $\mathcal{D}_q$ contains the latest M dates of $\mathcal{D}_q^\infty$; a fixed store contains its earliest M dates.

Fitting dates are selected independently of $\mathcal{D}_q$. Let

$$c_q = \begin{cases} q - P \lceil H/P \rceil, & \text{alignment on and } P \mid S_{\text{fit}}, \\ q - H, & \text{otherwise,} \end{cases}$$

where $P \mid S_{\text{fit}}$ means that S_{fit} is a multiple of P . The rolling fitting set is the latest N_{fit} dates in

$$\mathcal{F}_q^\infty = \{c_q - jS_{\text{fit}} : j \in \mathbb{N}_0, c_q - jS_{\text{fit}} \geq L - 1\}. \quad (5)$$

Here N_{fit} always counts dates per user. Same-user fitting uses N_{fit} rows for each user; all-user fitting uses $N_{\text{fit}}U$ rows for one shared fit. A fixed fit freezes the first complete $\mathcal{F}_q$ for each phase modulo S_{fit} .

The first evaluated date q_0 is the first date for which q_0 and every date in $\mathcal{F}_{q_0}$ have K_{max} retrievable neighbors and $|\mathcal{F}_{q_0}| = N_{\text{fit}}$. The evaluated dates are

$$\mathcal{Q} = \{q_0 + jS_q : j \in \mathbb{N}_0, q_0 + jS_q \leq T - H - 1\}. \quad (6)$$

2.2 Retrieval

Let $\phi(x) \in \mathbb{R}^J$ be the retrieval representation of a lookback, where J is its number of features, and let d be the configured distance. The default is $\phi(x) = x$ and

$$d(x, x') = \|x - x'\|_2. \quad (7)$$

For each (q, u) , exact search returns the K_{max} smallest-distance windows $(r_{q,u,k}, v_{q,u,k})$, $k \in \{1, \dots, K_{\text{max}}\}$, where $r_{q,u,k} \in \mathcal{D}_q$ is a date and $v_{q,u,k}$ is a user. The allowed users are all v , only $v = u$, or only $v \neq u$, according to retrieval scope.

Let $\mu(x)$ and $\sigma(x)$ be the mean and population standard deviation of the entries of x , and let $\varepsilon = 10^{-8}$. A value a attached to a neighbor lookback x' is put on the query scale by

$$R_x(a; x') = \frac{a - \mu(x')}{\max\{\sigma(x'), \varepsilon\}} \max\{\sigma(x), \varepsilon\} + \mu(x). \quad (8)$$

2.3 Signals used by the ridge

Let $f : \mathbb{R}^L \rightarrow \mathbb{R}^H$ be the frozen forecaster. For every (q, u) and neighbor rank k , define

$$\begin{aligned} V_{q,u} &= f(x_{q,u}), & \text{vanilla forecast,} \\ Y_{q,u,k} &= R_{x_{q,u}}(y_{r_{q,u,k}, v_{q,u,k}}; x_{r_{q,u,k}, v_{q,u,k}}), & \text{retrieved future,} \\ N_{q,u,k} &= R_{x_{q,u}}(f(x_{r_{q,u,k}, v_{q,u,k}}); x_{r_{q,u,k}, v_{q,u,k}}), & \text{forecast of the retrieved lookback.} \end{aligned}$$

For a chosen $K \leq K_{\text{max}}$, the distance-weighted retrieved future is

$$\begin{aligned} \bar{Y}_{q,u,K} &= \sum_{k=1}^K w_{q,u,k} Y_{q,u,k}, & w_{q,u,k} &= \frac{e^{-\delta_{q,u,k}}}{\sum_{j=1}^K e^{-\delta_{q,u,j}}}, \\ \delta_{q,u,k} &= \frac{d_{q,u,k} - \min_{j \leq K} d_{q,u,j}}{\max\{\text{sd}(d_{q,u,1:K}), \varepsilon\}}, \end{aligned} \quad (9)$$

where $d_{q,u,k}$ is the stored retrieval distance and sd is population standard deviation.

The code also defines a context forecast $C_{q,u,K}$ by calling the canonical backbone with the first K rescaled neighbor pairs translated into named covariates. With `retrieval_covariate_mode=past_and_future`, the past covariates are the neighbor lookbacks and the future covariates are their known horizons; `past` omits the latter and `none` disables the wrapper. Chronos-2 therefore computes

$$C_{q,u,K} = f\left(x_{q,u}; \tilde{X}_{q,u,1:K}, \tilde{Y}_{q,u,1:K}\right), \quad (10)$$

through the official pipeline. The model alias remains `chronos2` in all three modes. Passing context to a backbone that declares no covariate support raises an error; it is never replaced by $V_{q,u}$. The four-backbone profile uses the covariate-free `y_ridge_shared` design, while the primary Chronos-2 ridge and gate profiles use the context forecast.

For one date, user, and horizon position, the feature vector is selected from the following signals. Each Y or N entry denotes all ranks $1, \dots, K$.

Design name	Feature signals
cov	V, C
avgy	$V, \bar{Y}$
y	$V, Y_1, \dots, Y_K$
cov_y	$V, C, Y_1, \dots, Y_K$
cov_avgy	$V, C, \bar{Y}$
residual	$V, Y_1, \dots, Y_K, N_1, \dots, N_K$
full	$V, C, Y_1, \dots, Y_K, N_1, \dots, N_K$

The main method is `full_ridge_shared`. Let $D \in \mathbb{R}^{nH \times p}$ contain its $p = 2 + 2K$ features for n fitting date-user rows, and let $e \in \mathbb{R}^{nH}$ contain $y - V$. If the fitting loss is nMSE, each date-user row of D and e is first divided by $\max\{\sigma(x_{r,u}), \varepsilon\}$; for MSE it is divided by 1. Let

$$S = \text{diag}(s_1, \dots, s_p), \quad s_j = \max \left\{ \sqrt{\frac{1}{nH} \sum_{i=1}^{nH} D_{ij}^2}, 10^{-12} \right\}.$$

For ridge value $\alpha \geq 0$, the fitted coefficients are

$$\hat{\beta} = S^{-1} \left(\frac{S^{-1} D^\top D S^{-1}}{nH} + \alpha I_p \right)^{-1} \frac{S^{-1} D^\top e}{nH}, \quad (11)$$

where I_p is the $p \times p$ identity matrix. The prediction is

$$\hat{y}_{q,u} = V_{q,u} + D_{q,u} \hat{\beta}. \quad (12)$$

Same-user fitting gives one $\hat{\beta}$ per (q, u) ; all-user fitting gives one $\hat{\beta}$ per q . Shared ridge uses one coefficient vector for all H forecast positions. Horizon ridge fits one vector $\hat{\beta}_h$ for each $h \in \{1, \dots, H\}$.

Delta ridge removes V from D , replaces every remaining signal A by $A - V$, and still predicts (12). Convex fitting uses the unmodified signals, targets y , and solves

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{nH} \|y - D\beta\|_2^2 \quad \text{subject to} \quad \beta_j \geq 0, \quad \sum_{j=1}^p \beta_j = 1; \quad (13)$$

its prediction is $D\hat{\beta}$.

2.4 Selection at each query

Let $\rho \in (0, 1)$ be the validation ratio and $m = \lceil \rho N_{\text{fit}} \rceil$. When α or K is selected, the oldest $N_{\text{fit}} - m$ dates of $\mathcal{F}_q$ fit each candidate and the newest m dates score it. The default candidate sets are

$$\mathcal{A} = \{10^{-1}, 10^{-2}, 10^{-3}\}, \quad \mathcal{K} = \{1, 5, 10, 15\}, \quad K_{\max} = 20, \quad \rho = 0.2. \quad (14)$$

The selected pair minimizes validation MSE or nMSE. Exact ties choose smaller K , then larger α . The selected pair is refitted on all of $\mathcal{F}_q$. Supplying `used_k` fixes K but does not fix α ; setting `tune_alpha=False` fixes α but does not fix K .

2.5 Gate ablations and TS-RAG comparison

For a gate candidate $A_{r,u} \in \{C_{r,u,K}, \bar{Y}_{r,u,K}\}$, define its horizon-averaged advantage

$$a_{r,u} = \frac{1}{H} \sum_{h=1}^H \frac{(y_{r,u,h} - V_{r,u,h})^2 - (y_{r,u,h} - A_{r,u,h})^2}{c_{r,u}^2}, \quad (15)$$

where $c_{r,u} = 1$ for MSE fitting and $c_{r,u} = \max\{\sigma(x_{r,u}), \varepsilon\}$ for nMSE fitting. For the method named `bayes`, the score $s_{q,u}$ is the mean of $a_{r,u}$ over the fitting rows. For `catboost`, define $\Delta = A - V$, $E = Y_{1:K} - V$, $G = Y_{1:K} - N_{1:K}$, the rescaled neighbor lookbacks $X_k = R_{x_{r,u}}(x_{r_k, v_k}; x_{r_k, v_k})$, neighbor

ages $g_k = r - r_k$, and same-user indicators $o_k = \mathbf{1}[v_k = u]$. Means and population standard deviations below use every entry of the indicated array. The regressor input is

$$\psi_{r,u} = (\text{mean } \Delta, \text{sd } \Delta, \text{mean } E, \text{sd } E, \text{mean } G, \text{sd } G, \mu(x_{r,u}), \sigma(x_{r,u}), \\ \text{mean } X, \text{sd } X, \text{mean } o, \text{mean } g, \text{sd } g, \text{mean } d, \text{sd } d, \min d) \in \mathbb{R}^{16}. \quad (16)$$

CatBoost is fitted to map $\psi_{r,u}$ to $a_{r,u}$, and its output is $s_{q,u}$. Let $\tau_{q,u}$ be the population standard deviation of the fitting advantages. Both methods predict

$$\hat{y}_{q,u} = V_{q,u} + \text{sigmoid}(s_{q,u}/\tau_{q,u})(A_{q,u} - V_{q,u}), \quad (17)$$

where $\text{sigmoid}(x) = (1 + e^{-x})^{-1}$. When $\tau_{q,u} = 0$, the weight is 1, 0, or 1/2 for $s_{q,u} > 0$, $s_{q,u} < 0$, or $s_{q,u} = 0$. The current gate front uses $A = C$, $K = 20$, and backbones with (10); hence its predictions are exactly V under the current code.

The TS-RAG comparison is source-adapted from UConn-DSIS/TS-RAG commit 73ac807. It is fixed to $L = 512$, $H = 64$, $K = 5$, Chronos-Bolt, and same-user retrieval. Its retriever uses the final Chronos-T5 token $\phi(x)$, float32 `faiss.IndexFlatL2`, a $K + 1$ search, and removal of the final returned column. Its ARM receives the original, unscaled retrieved lookback–future sequences. Only the set of dates allowed by (1)–(4) differs from the released retrieval procedure.

3 Measures

For prediction $\hat{y}_{q,u}$ and query scale $s_{q,u} = \max\{\sigma(x_{q,u}), \varepsilon\}$, define

$$\text{MSE}_{q,u} = \frac{1}{H} \sum_{h=1}^H (y_{q,u,h} - \hat{y}_{q,u,h})^2, \quad \text{nMSE}_{q,u} = \frac{\text{MSE}_{q,u}}{s_{q,u}^2}, \quad (18)$$

$$\text{MAE}_{q,u} = \frac{1}{H} \sum_{h=1}^H |y_{q,u,h} - \hat{y}_{q,u,h}|, \quad \text{nMAE}_{q,u} = \frac{\text{MAE}_{q,u}}{s_{q,u}}. \quad (19)$$

Every aggregate is an unweighted mean over the retained date–user rows. Methods are compared only on their common query dates. If m denotes an adapted method and v its matching vanilla forecast, report

$$I_m^{\text{MSE}} = 100 \frac{\text{MSE}_v - \text{MSE}_m}{\text{MSE}_v}, \quad I_m^{\text{nMSE}} = 100 \frac{\text{nMSE}_v - \text{nMSE}_m}{\text{nMSE}_v}, \quad (20)$$

and the strict win rate

$$W_m = 100 \frac{1}{|\mathcal{Q}|U} \sum_{q \in \mathcal{Q}} \sum_{u=1}^U \mathbf{1}[\text{MSE}_{m,q,u} < \text{MSE}_{v,q,u}], \quad (21)$$

where $\mathbf{1}[\cdot]$ is 1 when its condition is true and 0 otherwise. The report also saves population standard deviation across per-user means, the mean over the worst $\lceil 0.1U \rceil$ users selected separately for MSE and nMSE, and the percentage of users improved.

Reported time is extraction time plus adaptation/evaluation time. It is also divided by $|\mathcal{Q}|U$. A separate cold-batch value measures the first all-user query with empty in-memory caches.

4 Runs

The default ridge run uses raw Euclidean all-user retrieval, same-user fitting, rolling store and fit, $N_{\text{store}} = 30,000$, $N_{\text{fit}} = 100$, $S_{\text{store}} = S_{\text{fit}} = P$, seed 1, and (14). Test runs use $S_q = 257$; small and full runs use $S_q = 127$. The frozen backbone is Chronos-2. The main family also adds the TS-RAG configuration defined above.

Mode	Datasets and (L, H) settings
test	Electricity, long range only
small	Electricity, Solar, and Traffic; short, mid, and long ranges
full	small plus raw ETTh1, ETTh2, ETTm1, ETTm2 and every prepared TIME dataset; short, mid, and long ranges

Frequency	short	mid	long
hourly	(168, 24)	(336, 48)	(504, 168)
daily	(7, 1)	(14, 2)	(30, 7)
15-minute	(96, 4)	(192, 8)	(672, 96)

Focused DGX fronts under `slurm/dgx/ablations/` change only the listed values; their Selena counterparts are under `slurm/selena/ablations/`:

Front	Values
<code>ablation_n_store.slurm</code>	test: {1000, 2000, 5000}; publication: {10000, 20000, 30000, 50000}
<code>ablation_n_fit.slurm</code>	test: {50, 100, 500}; publication adds 1000
<code>ablation_fit_stride.slurm</code>	$S_{\text{fit}} \in \{1, P\}$
<code>ablation_alpha.slurm</code>	fixed $\alpha \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$; select K
<code>ablation_k.slurm</code>	fixed $K \in \{1, 3, 5, 10, 15, 20\}$; select α
<code>ablation_l.slurm</code>	$H = 24$, $L \in \{24, 96, 168, 336, 504, 512, 720\}$
<code>ablation_h.slurm</code>	$L = 504$, $H \in \{24, 48, 64, 96, 168, 336, 504\}$
<code>ablation_feature_design.slurm</code>	the seven designs in the signal table
<code>ablation_formulation.slurm</code>	ridge, delta ridge, and convex; shared and horizon forms
<code>ablation_fixed_protocol.slurm</code>	rolling/fixed store crossed with rolling/fixed fit
<code>ablation_general_scope.slurm</code>	all/same/other-user retrieval crossed with all/same-user fitting
<code>ablation_homogeneous.slurm</code>	all channels versus declared same-quantity channels on ETT and Weather
<code>ablation_backbones.slurm</code>	Chronos-2, Chronos-Bolt, Chronos-T5, and TS-ICL at (512, 64)
<code>ablation_sota_chronos_bolt.slurm</code>	causal Chronos-Bolt at (512, 64) beside published TS-RAG/Cross-RAG rows; protocols are labeled non-comparable

The gate front evaluates `bayes_cov_shared_soft` and `catboost_cov_shared_soft`. CatBoost uses 300 trees in small/full and 2 trees in test, depth 4, learning rate 0.03, and refit stride 1.

5 Code

5.1 Data objects

Let $W = T - L - H + 1$ be the number of valid window dates, R the number of unique query and fitting dates that require retrieval, B the number of unique window IDs that require a vanilla forecast, and J the retrieval-feature dimension. The source CSV is loaded once as `CsvTimeSeries`; overlapping lookbacks and targets remain sliding views of Z .

Object	Shape or key	Stored value and code owner
<code>tables/windows</code>	W dates, U users	dates, timestamps, user names; <code>src/proposal/extraction.py::_window_metadata</code>
<code>tables/window_statistics</code>	$W \times U$	lookback mean, standard deviation, constant flag; same owner
<code>tables/window_computation</code>	optional $A \times U \times J$ and $B \times H$	compact retrieval representations and forecasts, where A is the number of dates requiring a representation; <code>src/pipeline/extraction.py::extract_online_features</code>
<code>tables/retrieval_neighbors</code>	$R \times U \times K_{\max}$	neighbor window ID and configured/raw/normalized distances; also R dates, query flags, and candidate counts; same owner
<code>context_cache/K*/window*.npy</code>	$(r, K) \mapsto \mathbb{R}^{U \times H}$	requested context forecasts only; <code>src/proposal/extraction.py::ContextForecastCache</code>
<code>selected_hyperparameters.csv</code>	(q, u) or q	selected α , K , validation loss, parameter count; <code>src/pipeline/adaptation.py::evaluate_online_linear</code>
<code>coefficient_trajectory.npy</code>	query, optional user, optional horizon, feature	refitted ridge coefficients; same owner
<code>per_user_date_metrics.csv</code>	(q, u, method)	(18)–(21) and vanilla values; <code>src/proposal/ridge.py</code>
<code>metrics.json</code> , <code>compute_timing.json</code>	one result	aggregate measures and complete/cold times; <code>src/results/efficiency.py</code> and <code>src/pipeline/adaptation.py</code>

A stable window ID is

$$\text{id}(r, u) = \text{pos}(r)U + (u - 1), \quad (22)$$

Here $\text{pos}(r)$ is the zero-based position of date r in the valid-date array, so $0 \leq \text{pos}(r) < W$. `open_extraction_arrays` in `src/pipeline/contracts.py` reconstructs source windows, retrieved values, and sparse forecast joins from these IDs.

5.2 Pipeline

Algorithm 1 Extraction

Require: panel Z ; settings $L, H, N_{\text{store}}, N_{\text{fit}}, S_{\text{store}}, S_{\text{fit}}, S_q, K_{\text{max}}$

- 1: build sliding views $x_{r,u}, y_{r,u}$ and store their IDs, means, and scales
- 2: build $\mathcal{Q}$ from (6)
- 3: **for all** $q \in \mathcal{Q}$ **do**
- 4: build $\mathcal{F}_q$ from (5)
- 5: **end for**
- 6: $\mathcal{R} \leftarrow \mathcal{Q} \cup \bigcup_{q \in \mathcal{Q}} \mathcal{F}_q$
- 7: **for all** $r \in \mathcal{F}_{q_0} \cup \{q_0\}$ first, then remaining $r \in \mathcal{R}$ by date **do**
- 8: build $\mathcal{D}_r$ from (3)–(4)
- 9: **for** $u = 1$ to U **do**
- 10: exact-search the allowed users and save ranks $1:K_{\text{max}}$
- 11: **end for**
- 12: **end for**
- 13: forecast each query or selected-neighbor window ID once
- 14: save extraction arrays, diagnostics, manifest, and time

Algorithm 2 Adaptation and evaluation

Require: extraction arrays; method; fitting scope; candidate α and K values

- 1: **for all** $q \in \mathcal{Q}$ in chronological order **do**
- 2: reconstruct $\mathcal{F}_q$ from (5)
- 3: **if** method is ridge, delta ridge, or convex **then**
- 4: build the chosen design from $V, C, \bar{Y}, Y_{1:K}, N_{1:K}$
- 5: select α, K on the newest validation dates when enabled
- 6: refit on all $\mathcal{F}_q$ with (11) or (13)
- 7: predict with (12) or $D\hat{\beta}$
- 8: **else if** method is a gate **then**
- 9: fit the mean-advantage or CatBoost score from (15)
- 10: predict with (17)
- 11: **else**
- 12: run TS-RAG ARM on its raw retrieved sequences
- 13: **end if**
- 14: save (18)–(19) for every user and the all-user date batch
- 15: **end for**
- 16: aggregate (20)–(21), user measures, coefficients/importances, and time

The direct code path is

`src/scripts/extract.py` $\rightarrow$ `src/pipeline/extraction.py` $\rightarrow$ `src/scripts/adapt.py` $\rightarrow$
`src/pipeline/adaptation.py` $\rightarrow$ `src/scripts/tables.py`.

Date selection is in `src/proposal/datastore.py`; ridge designs and solves are in `src/proposal/ridge.py`; gates are in `src/proposal/gates.py`; TS-RAG is in `src/external_models/tsrag/` and `src/pipeline/tsrag.py`; run lists are defined only in `src/pipeline/profiles.py`.

6 Execution

Each DGX front under `slurm/dgx/` has a matching `_selena.slurm` overflow front under `slurm/selena/`. Both set the same family and call `src/slurm/run_family.sh`. The default order is `extract, adapt, tables`; STAGES may contain a recovery subset. Each stage uses Hydra and the same seed. Completed runs with the exact recorded settings are reused. Training and full inference run only on the cluster. DGX arrays, metrics, figures, and tables are under `outputs/`, and run logs are under `logs/`. Selena fronts preserve the same relative artifact identity under `outputs_selena/`

and write their streams under `logs_selena/`, so returned overflow work never collides with DGX trees. Selena uses partition `an` with QoS `an_preemptable`.

No online-adaptation result has yet been analyzed. This document therefore defines computations and runs only; it makes no performance claim.